

2.1 System Architecture Design

Project: E-Learning Web Pages for Computer Science

Student: Zain Chohan (32001784)

Supervisor: Richard Mitchell

Overview

The system is designed as a **modular, web-based application** that simulates a simplified 5-stage CPU instruction pipeline.

The architecture follows a **Model–View–Controller (MVC)** pattern to separate concerns between simulation logic, visualisation, and user interaction.

This design ensures **maintainability, scalability**, and a clear relationship between backend logic (simulation) and frontend representation (user interface).

Architecture Layers

Layer	Component	Responsibility
Model	CPU, Memory, Registers, Pipeline Buffers	Manages all internal data structures and simulation state.
View	HTML5 Canvas / SVG Visualiser	Renders pipeline stages, instruction flow, and hazards in real time.
Controller	Simulation Engine, Event Handlers	Executes the pipeline algorithm, responds to user inputs (play, step, reset).

System Components

1. User Interface Layer

- Displays visual pipeline diagram (IF → ID → EX → MEM → WB).
- Contains user controls:
Play, Pause, Step, Reset, Speed Control.

- Provides text area for assembly input and output metrics (CPI, total cycles, hazards).

2. Simulation Engine

- Core JavaScript module that performs the CPU pipeline logic.
- Handles stage transitions, clock cycles, and hazard detection.
- Implements stalling, forwarding, and branch prediction.

3. Pipeline State Manager

- Maintains pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB).
- Updates each stage's state on every clock cycle.
- Manages dependencies between instructions.

4. Visualisation Module

- Uses Canvas API or SVG (via D3.js) to draw and animate instruction movement.
- Updates in synchrony with the simulation clock.
- Displays hazards visually (e.g., red highlights for stalls, arrows for forwarding).

5. Performance and Metrics Module

- Calculates and displays metrics:
 - Total Cycles
 - CPI (Cycles Per Instruction)
 - Stalls and Forwarding Count
 - Branch Accuracy
- Graphically represented using Chart.js.

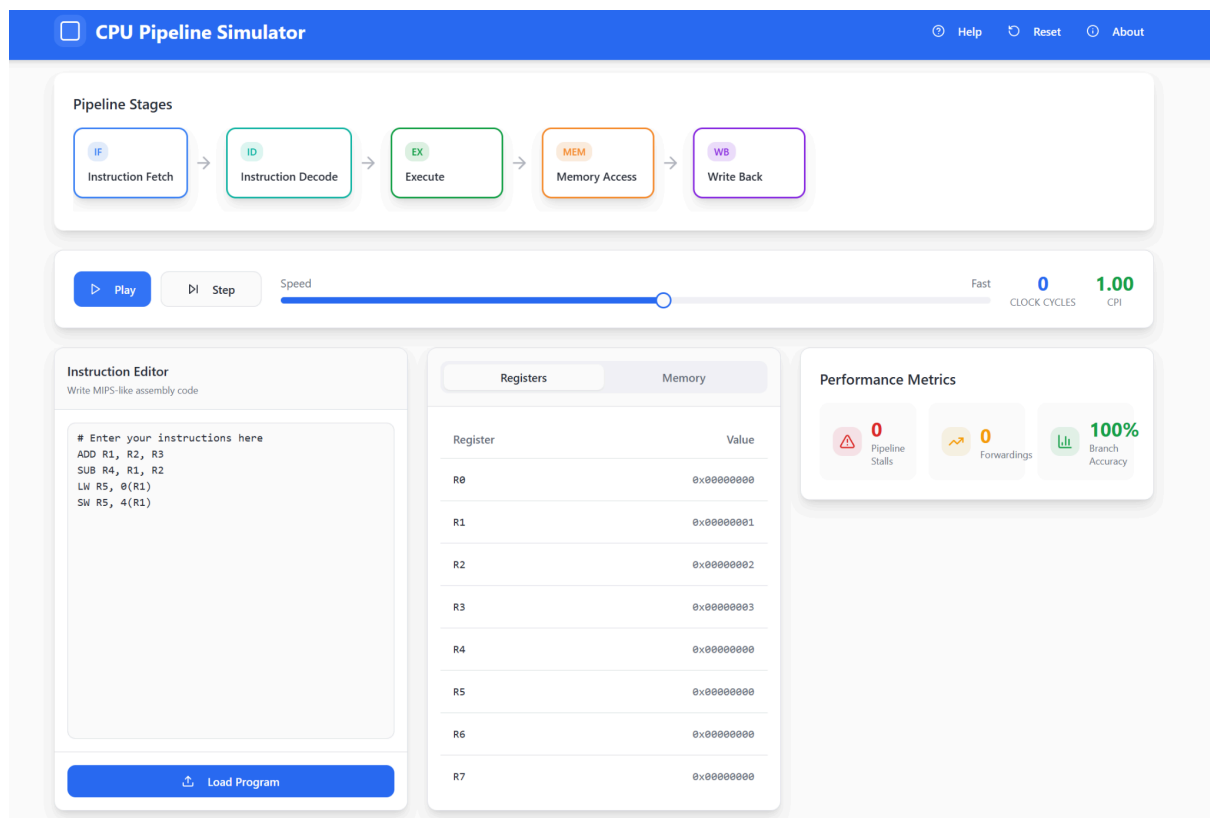
6. SLEA Evaluation Module

- Hosts a short questionnaire to assess user learning and usability.

- Stores responses locally and anonymises data per University ethical guidelines.

System Data Flow

1. User inputs MIPS-like code via the editor.
2. The controller parses instructions and initialises the pipeline model.
3. The simulation engine executes one cycle per user action (step) or at timed intervals (play).
4. Pipeline states are updated and sent to the view for rendering.
5. Visualisation module animates instruction progress and hazards.
6. The metrics module records performance data and updates dashboard charts.



Design Justification

- **Web-based modular design** allows platform independence and simple deployment.
- **MVC pattern** ensures a clear separation between data logic and interface rendering.
- **Component-based JS structure (optional React integration)** allows scalability for future features such as superscalar pipelines or multi-core extensions.
- **Accessibility-first design** ensures compliance with WCAG 2.1 standards for educational use.